



Proyecto de Integración Final

ICC708-1: PROGRAMACION AVANZADA

Nombre: Bárbara Constanza Sanhueza Bombín.

Carrera: Ingeniería informática.

Fecha: 10-07-2026.

Docente: Profesor Ricardo Gacitúa B.

Diseño de los ADT

Al revisar la implementación, se identifican **cinco ADT principales**: Incident, EmergencyCenter, RoadNetwork, IncidentHashTable e IncidentPriorityQueue. Aunque el enunciado menciona explícitamente los tres primeros como modelo de datos, la tabla hash y la cola de prioridad también corresponden a tipos abstractos de datos, ya que mantienen un estado interno y ofrecen operaciones definidas sobre él. IncidentSorter y RouteFinder no se consideran ADT, porque son componentes algorítmicos sin estado propio.

ADT Incident

Incident representa una solicitud de emergencia recibida por el sistema. Almacena el identificador, la zona, la ubicación geográfica, la prioridad, el tipo de emergencia, la fecha de reporte y el estado actual.

Sus datos deben cumplir ciertas condiciones: el identificador y los textos no pueden estar vacíos, la ubicación debe contener latitud y longitud numéricas, la prioridad debe ser Baja, Media, Alta o Crítica, y la fecha debe utilizar el formato AAAA-MM-DD HH:MM:SS. Todo incidente se crea inicialmente con estado Pendiente.

Las operaciones principales son crear el incidente, actualizar su estado, cambiar su prioridad y convertir la fecha almacenada en un objeto de tiempo. Este ADT es utilizado posteriormente por la tabla hash, el Max-Heap y los algoritmos de generación de reportes.

ADT EmergencyCenter

EmergencyCenter representa un centro desde el cual se despachan los equipos de respuesta. Su información se compone de un identificador, un nombre y una ubicación correspondiente a un nodo de la red vial.

El identificador y el nombre deben ser textos no vacíos, mientras que la ubicación debe contener dos valores numéricos. Su operación principal es la creación y validación del centro. Una vez creado, el objeto se utiliza como punto de origen para calcular rutas hacia los incidentes.

En el escenario implementado se utilizan tres centros de emergencia ubicados en diferentes nodos del grafo. Esto permite calcular una ruta desde cada centro y seleccionar aquel que tenga el menor costo de desplazamiento.

ADT RoadNetwork

RoadNetwork representa la red vial como un grafo dirigido y ponderado. Los nodos corresponden a intersecciones o localidades, mientras que las aristas representan calles. Cada arista posee un peso que indica el costo o tiempo estimado de desplazamiento.

Internamente se utiliza una lista de adyacencia, donde cada coordenada se relaciona con una lista de nodos vecinos y sus respectivos pesos. Los pesos deben ser positivos, los nodos deben existir antes de crear una calle y no se permiten aristas duplicadas entre un mismo origen y destino.

Sus operaciones permiten agregar intersecciones, agregar calles, consultar los vecinos de un nodo, comprobar la existencia de una intersección y obtener la cantidad de nodos y aristas. También puede cargar una red desde un archivo JSON. Este ADT es la estructura sobre la cual se ejecutan BFS, Dijkstra y A*.

ADT IncidentHashTable

IncidentHashTable almacena los incidentes utilizando su identificador como clave. La tabla fue desarrollada manualmente mediante una lista de buckets y utiliza encadenamiento separado para resolver colisiones, sin emplear un diccionario de Python como tabla hash.

La posición de cada incidente se calcula mediante una función hash polinomial. Si dos identificadores producen el mismo índice, ambos quedan almacenados dentro del mismo bucket. Cuando se intenta insertar un identificador existente, el incidente anterior se reemplaza en vez de generar un duplicado.

Sus operaciones permiten insertar, buscar, actualizar el estado y eliminar incidentes. Además, entrega estadísticas sobre el factor de carga, el número de colisiones, los buckets utilizados y el tamaño del bucket más grande.

ADT IncidentPriorityQueue

IncidentPriorityQueue administra los incidentes según su nivel de urgencia. Se implementa mediante un **Max-Heap propio**, almacenado en una lista, donde el incidente más urgente permanece en la raíz.

La urgencia no depende únicamente de la prioridad original. Se calcula mediante la expresión:

urgencia = severidad × (1 + horas de espera)

Las prioridades se convierten en valores numéricos: Baja vale 1, Media 2, Alta 3 y Crítica 4. De esta forma, un incidente puede aumentar su urgencia mientras permanece esperando.

Las operaciones principales son insertar un incidente, extraer el más urgente, actualizar una prioridad, consultar la urgencia y mostrar los k incidentes más críticos sin eliminarlos. Después de cada modificación se aplican operaciones de subida o bajada para mantener la propiedad del Max-Heap.

Componentes algorítmicos complementarios

IncidentSorter no mantiene información interna, sino que reúne las implementaciones manuales de QuickSort y MergeSort utilizadas para ordenar incidentes por fecha, prioridad o zona. RouteFinder tampoco almacena un estado propio; recibe una red vial y ejecuta BFS, Dijkstra o A* para obtener los nodos visitados, la ruta encontrada, la distancia y el costo acumulado. Por esta razón, ambos se consideran clases de apoyo algorítmico y no ADT del sistema.

Diagramas

Diagrama de flujo.

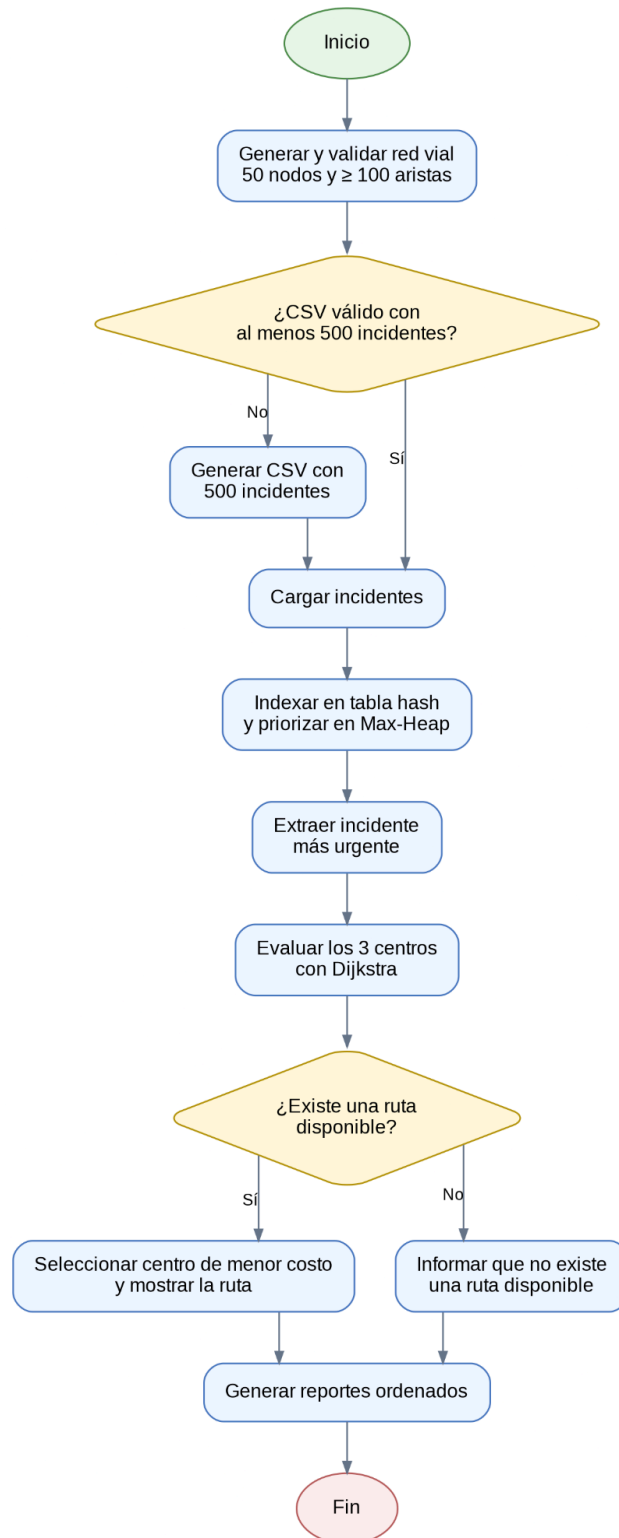
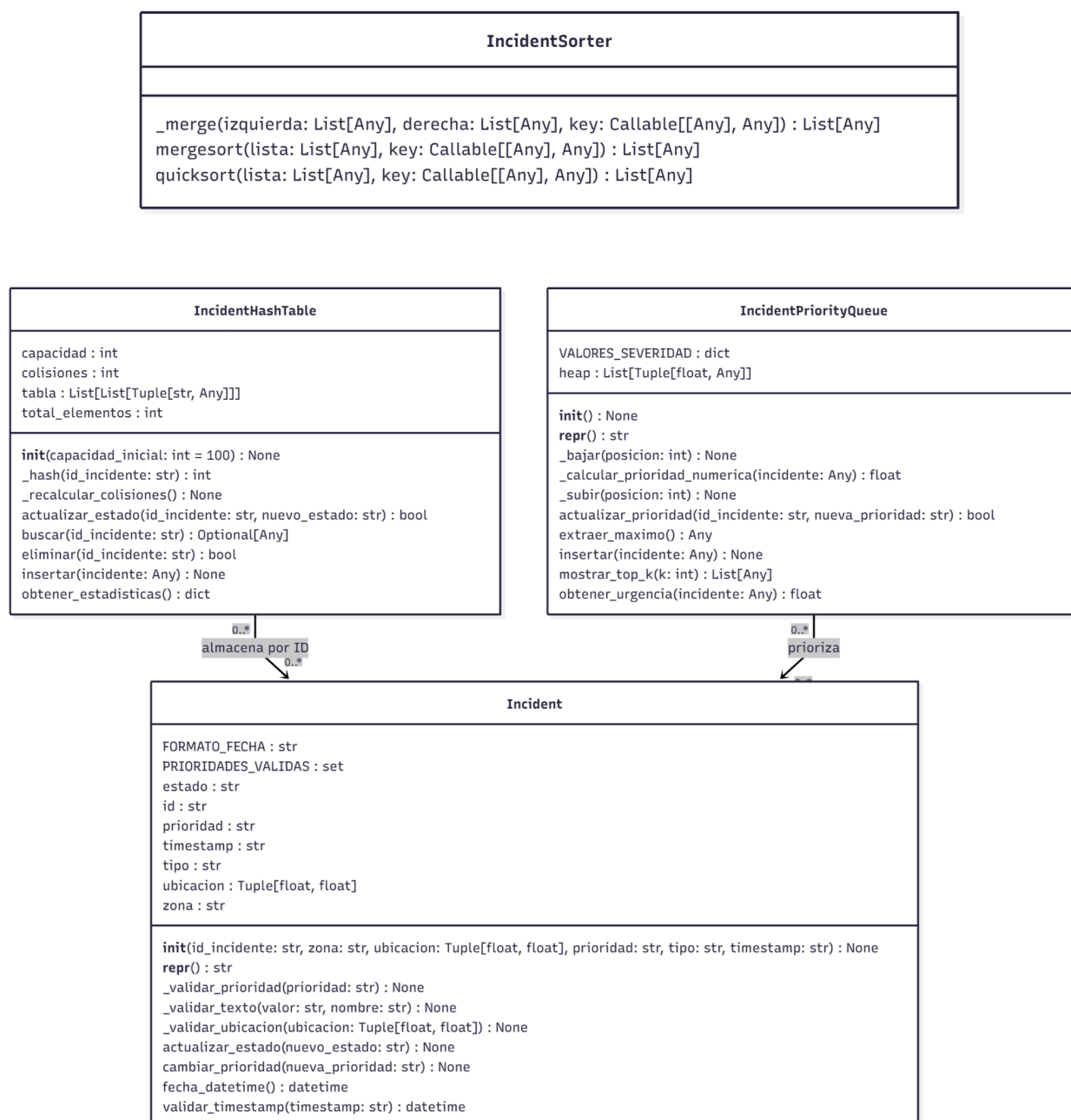
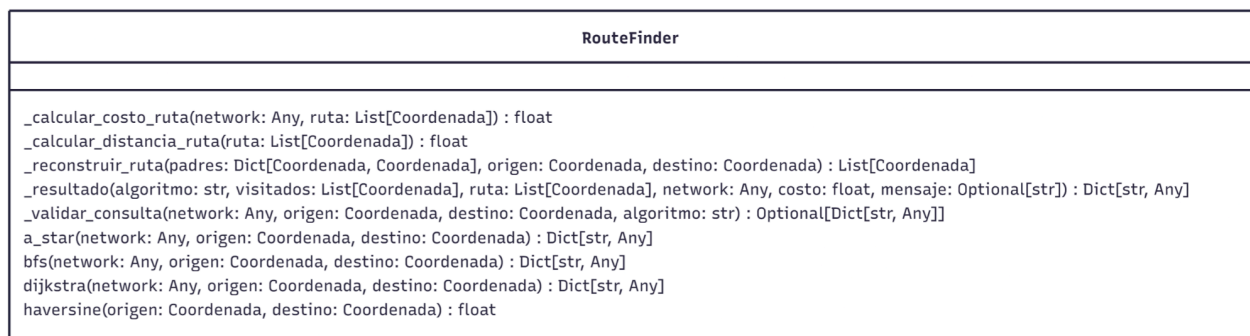
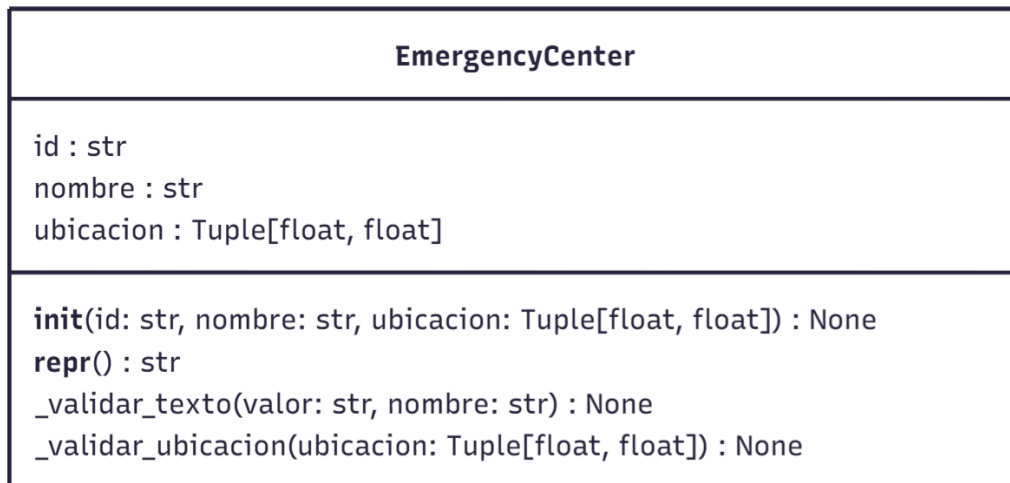
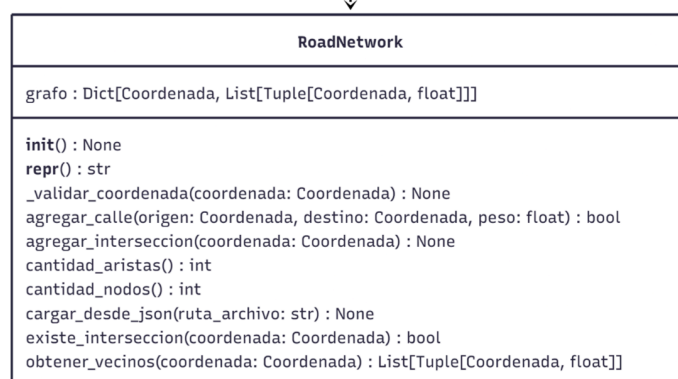


Diagrama UML.





utiliza



Complejidades

El análisis consideran incidentes, **m** buckets de la tabla hash, **V** nodos, **E** aristas, **d** vecinos de un nodo y **k** elementos solicitados en el Top-K. Las complejidades corresponden a las operaciones principales de la implementación.

Estructura o algoritmo	Operación	Complejidad temporal	Complejidad espacial
Incident y EmergencyCenter	Crear o modificar datos	$O(1)$	$O(1)$
RoadNetwork	Agregar nodo / consultar vecinos	$O(1)$	$O(1)$
RoadNetwork	Agregar calle	$O(d)$	$O(1)$
Tabla hash	Buscar o actualizar estado	$O(1)$ promedio; $O(n)$ peor caso	$O(1)$
Tabla hash	Insertar o eliminar	$O(m)$ en la implementación actual*	$O(1)$
Max-Heap	Insertar / extraer más urgente	$O(\log n)$	$O(1)$
Max-Heap	Actualizar prioridad / mostrar Top-K	$O(n)$ / $O(k \cdot n)$	$O(1)$ / $O(k)$
QuickSort	Ordenar incidentes	$O(n \log n)$ promedio; $O(n^2)$ peor caso	$O(n \log n)$ promedio
MergeSort	Ordenar incidentes	$O(n \log n)$ en todos los casos	$O(n)$
BFS	Buscar una ruta sin considerar pesos	$O(V + E)$	$O(V)$
Dijkstra y A*	Buscar ruta de menor costo	$O((V + E) \log V)$	$O(V + E)$

La inserción y eliminación de la tabla hash tendrían costo promedio cercano a $O(1)$, pero el código recalcula las colisiones recorriendo todos los buckets después de cada modificación, lo que agrega un costo $O(m)$. El Max-Heap mantiene inserción y extracción en $O(\log n)$, mientras que la actualización requiere buscar primero el incidente dentro de la lista.

Resultados experimentales

Las pruebas se realizaron con un conjunto de 500 incidentes, una red vial de 50 nodos y 100 aristas dirigidas, y tres centros de emergencia. Para medir los tiempos se utilizó `time.perf_counter()`. En los algoritmos de ordenamiento se probaron listas de 100, 250 y 500 elementos, en estado aleatorio, ordenado e invertido, con 20 repeticiones por caso.

Elemento evaluado	Resultado
Incidentes procesados	500
Incidente más urgente	I-204, prioridad Crítica, urgencia 803,60
Centro seleccionado	C-003 – Centro Sur
Ruta sugerida	3 nodos y 2 tramos
Distancia total	12,063 km
Costo y tiempo estimado	17,4 minutos
Zona con más incidentes	Zona_11, con 36 casos

El sistema logró cargar, priorizar y asignar correctamente el incidente más urgente. Para el incidente I-204 se evaluaron los tres centros mediante Dijkstra y se seleccionó el Centro Sur, debido a que presentó la ruta disponible de menor costo.

Prueba	Resultado
Inserción de 500 incidentes en hash	0,040904 s
Búsqueda promedio en hash	0,000001208 s
Factor de carga	0,7118
Colisiones	60
Buckets utilizados	439

Tamaño máximo de bucket	2
Inserción de 500 incidentes en heap	0,008502 s
Extracción del máximo	0,000018500 s
Actualización de prioridad	0,000067800 s
Obtención del Top-10	0,000545900 s

La tabla hash encontró correctamente los 500 identificadores existentes y permitió actualizar y eliminar un incidente. El bajo tamaño máximo de los buckets indica que la función hash distribuyó los datos de forma adecuada. La pequeña diferencia con el factor de carga inicial de 0,7133 se debe a que durante la prueba se eliminó un incidente. En el Max-Heap, el elemento extraído coincidió con el incidente de mayor urgencia, validando la propiedad de prioridad.

Distribución de entrada	QuickSort promedio	MergeSort promedio
Aleatoria	0,002376915 s	0,001762870 s
Ordenada	0,001441465 s	0,001065070 s
Invertida	0,001177175 s	0,000985380 s

MergeSort obtuvo el menor tiempo promedio en los tres casos de 500 elementos y presentó resultados más estables. QuickSort también ordenó correctamente, pero mostró una mayor variación en el caso aleatorio, alcanzando un máximo de 0,011612300 segundos. La prueba básica adicional confirmó que ambos algoritmos generaron resultados correctos.

Algoritmo	Tiempo	Nodos visitados	Tramos	Costo
BFS	0,000045300 s	14	4	21,6
Dijkstra	0,000030800 s	12	4	21,6
A*	0,000034100 s	12	4	21,6

Los tres algoritmos encontraron la misma ruta en la consulta evaluada. Sin embargo, Dijkstra presentó el menor tiempo y visitó dos nodos menos que BFS. A* produjo el mismo comportamiento que Dijkstra porque utiliza una heurística igual a cero. En conjunto, los resultados

confirman que las estructuras implementadas funcionan correctamente y que Dijkstra es la alternativa más conveniente para seleccionar rutas considerando los pesos de la red vial.

Discusión

Las estructuras elegidas son adecuadas para las necesidades principales del sistema. La tabla hash permite localizar incidentes rápidamente mediante su identificador, mientras que el Max-Heap facilita extraer el incidente con mayor urgencia sin ordenar toda la colección. La lista de adyacencia resulta apropiada para una red vial con una cantidad limitada de conexiones, ya que utiliza espacio proporcional a los nodos y aristas existentes. Para las rutas, Dijkstra es la alternativa más apropiada porque los caminos poseen pesos positivos y el objetivo es minimizar el costo acumulado. BFS sirve como comparación, pero no garantiza la ruta de menor tiempo porque ignora los pesos.

La implementación presenta algunas oportunidades de mejora. Las colisiones de la tabla hash podrían actualizarse de forma incremental, evitando recorrer todos los buckets después de cada inserción o eliminación. La cola de prioridad podría mantener un índice auxiliar por ID para reducir la búsqueda necesaria al actualizar prioridades, y el Top-K podría obtenerse mediante una copia del heap en $O(k \log n)$. Además, la red utilizada es simulada y sus pesos representan tiempos aproximados. A* utiliza una heurística igual a cero, por lo que se comporta prácticamente como Dijkstra; con datos viales reales podría utilizarse una heurística geográfica admisible para explorar menos nodos sin perder la ruta óptima.